

**SRI DHARMASTHALA MANJUNATHESHWARA COLLEGE
(AUTONOMOUS), UJIRE - 574240**

**DEPARTMENT OF B.Voc
IN
SOFTWARE AND APPLICATION DEVELOPMENT
2024-2025**



**A PROJECT REPORT ON
“AI-POWERED VIRTUAL TRY-ON KIOSK”**

SUBMITTED BY:

CHARAN	231303
SINCHAN	231336
VARUN KUMAR JAIN	231337
ADESH JAIN	231342
KAUSHAL	2313

**UNDER THE GUIDANCE OF
Ms. NISHMITHA
DEPARTMENT OF B.Voc
IN
SOFTWARE AND APPLICATION DEVELOPMENT**

SRI DHARMASTHALA MANJUNATHESHWARA COLLEGE
(AUTONOMOUS), UJIRE - 574240

DEPARTMENT OF B.Voc
IN
SOFTWARE AND APPLICATION DEVELOPMENT
2024-2025



Certificate for the approval of project report

This is to certify that following students of III year B.Voc Software and Application Development has satisfactorily completed the project “**AI-POWERED VIRTUAL TRY-ON KIOSK**” for the B.Voc Software and Application Development. Prescribed by the college during academic year 2025-26.

Team Members,

1. CHARAN
2. SINCHAN
3. ADESH JAIN
4. VARNUN KUMAR JAIN
5. KOUSHAL

Head of the Department

(Ms. Nishmitha)
Project Guide

(Prof. Vishwanatha P)
Principal

Examiners

- 1
- 2

DECLARATION

We hereby declare that this project work entitled “**AI-POWERED VIRTUAL TRY-ON KIOSK**” has been prepared by us during the year 2025– 26 under the guidance of **Ms. Nishmitha** Department of **Software and Application Development**, SDM College in the partial fulfilment of B.Voc degree prescribed by the college.

We also declare that this project is the outcome of our own efforts, that it has not been submitted to any other university for the award of any degree.

Team Members

CHARAN	231303	
SINCHAN	231336	
VARUN KUMAR JAIN	231337	
ADESH JAIN	231342	
KAUSHAL	2313	

ACKNOWLEDGEMENT

We would like to thank our principal **Prof. Vishwanatha P**, for his support. We would like to thank HOD, Department of B.Voc in Software and application development, SDM College Ujire, for his valuable suggestions. We also like to thank **Ms. Nishmitha**, for her guidance.

Team Members

CHARAN	231303	
SINCHAN	231336	
VARUN KUMAR JAIN	231337	
ADESH JAIN	231342	
KAUSHAL	2313	

INDEX

SL.NO	TITLE	PAGE NO.
1.	Synopsis	1
	1.1 Title of the project	
	1.2 Description about the project	
	1.3 Project category	
	1.4 Project framework	
2.	Introduction	2
	2.1 Objective	
	2.2 Scope	
3.	Requirement Specification	3-6
	3.1 Hardware Requirement	
	3.2 Software Requirement	
	3.3 Tools and Technology	
	3.3.1 Next.js & React	
	3.3.2 SQLite Server	
	3.3.3 FastAPI / Python	
4.	3.3.4 VS Code	7-17
	3.3.5 FLUX.2 Klein 9B & ComfyUI	
	System Design Specification	
	4.1 Design Approach	
	4.2 Modules used in Application Design	
	4.3 Identification module description	
	4.3.1 Admin	
	4.3.2 Kiosk User	
	4.3.3 AI Orchestrator	
	4.4 Use case diagram	
	4.4.1 Admin	
	4.4.2 User	
	4.4.3 AI Orchestrator	
	4.5 DFD	
	4.6 ERD	
4.7 Database Design		
4.7.1 Introduction		
4.7.2 Database used in the module Implementation		
4.7.3 Pictorial representation of Database		

5.	Snapshots	18-28
6.	Source Code	29-85
7.	Conclusion	86
	7.1 Future Enhancement	
8.	Bibliography	87

CHAPTER - 1

SYNOPSIS

1.1 Title of the project:

“AI-POWERED VIRTUAL TRY-ON KIOSK”

1.2 Description about the project:

The main purpose of our project is to modernize the retail fashion experience by providing an interactive virtual try-on kiosk. Using an advanced generative AI model (FLUX.2 Klein 9B), customers can seamlessly visualize how different garments look on them without needing a physical trial room. The system handles live image capture, outfit recommendations based on gender and style, and generates photorealistic AI-driven cloth swapping in real-time.

1.3 Project Category:

Web Application & Generative AI Pipeline

1.4 Project Framework:

- Frontend Core: Next.js (React), TypeScript, Tailwind CSS
- Backend API: FastAPI (Python)
- AI Orchestration & Runtime: ComfyUI

CHAPTER - 2

INTRODUCTION

The traditional clothing trial process is time-consuming, unhygienic, and creates inefficiencies in retail stores. Many customers leave without purchasing due to crowded trial rooms or inability to visualize a garment's fit. Our AI-Powered Virtual Try-On Kiosk resolves these issues by allowing users to try on clothes digitally in real-time..

2.1 Objective

To develop a high-performance, user-friendly kiosk system that leverages generative AI for highly realistic virtual cloth swapping. The system aims to personalize the shopping experience by providing garment recommendations based on gender, occasion, and style, while significantly reducing the time spent in trial rooms..

2.2 Scope

The scope of this project includes replacing traditional trial rooms with an interactive digital kiosk. It is designed for retail usage but the architecture allows for a seamless transition into e-commerce web platforms. The solution covers a full-stack implementation involving a frontend kiosk interface, a local backend for data logic, and a cloud-based GPU instance running an advanced generative AI diffusion model (FLUX.2 Klein).

CHAPTER - 3

REQUIREMENT SPECIFICATION

Requirements specification is a specification of software requirements and hardware requirements required to do the project. Requirements analysis encompasses those tasks that go into determining the needs or conditions to meet for a new or altered product or project. Taking account of the possibly conflicting requirements of the various stakeholders, analyzing, documenting, validating and managing software or system requirements.

3.1 Hardware Requirements

SL. No	Hardware/Equipment	Specification
1	RAM	8GB
2	Processor	Intel Core i5+
3	Hard Disk	Minimum of 80 GB
4	Speed	2.8 GHz+

3.2 Software Requirements

Sl. No	Software	Specification
1	Operating System	Windows 10 or Higher
2	Language Used	TypeScript, Python
3	IDE	Visual Studio code
4	Database	SQLite

3.3 Tools and Technology

3.3.1 Next.js & React

Next.js 14 is a modern React framework used for building the user interface of our kiosk. It provides fast rendering, excellent developer experience, and simple routing. We utilized Tailwind CSS and Framer Motion for responsive design and premium animations. React's component-based architecture allowed for reusable UI elements across the kiosk experience.

3.3.2 SQLite Server

SQLite is a lightweight, serverless relational database management system. It was chosen to seamlessly store all garment metadata, categories, images, and pricing information locally on the kiosk terminal for ultra-fast load times. It integrates directly with our FastAPI backend using SQLAlchemy as the ORM.

3.3.3 FastAPI (Python)

FastAPI is an asynchronous Python web framework used to build our backend. It is extremely fast, highly scalable, and comes with built-in data validation using Pydantic. The backend handles image processing, database queries, and orchestrates the complex API requests sent to the remote AI server.

3.3.4 Visual Studio Code

Visual Studio Code (VS Code) is a lightweight yet powerful source code editor developed by Microsoft. It is widely used by developers for various programming languages, including JavaScript, Python, Java, C++, and more. VS Code provides a feature-rich development environment with functionalities such as syntax highlighting, intelligent code completion, debugging, Git integration, and extensions to enhance productivity. Whether you are a beginner or an experienced developer, VS Code offers a seamless coding experience across different platforms. As a tech enthusiast, having a basic understanding of VS Code can greatly improve your development workflow. In this article, we will guide you through the process of setting up and using Visual Studio Code on your computer.

3.3.5 FLUX.2 Klein 9B & ComfyUI

FLUX.2 Klein is a cutting-edge 9-billion parameter text-to-image AI model. Combined with a custom LoRA (Low-Rank Adaptation) trained specifically for virtual try-on, it produces photorealistic cloth swapping without complex segmentation masks. ComfyUI acts as the node-based backend engine to run the FLUX model, creating a stable pipeline connected via a REST JSON API.

CHAPTER - 4

SYSTEM DESIGN SPECIFICATION

4.1 Design Approach

This project requires a distributed architecture approach. The frontend and backend run locally on the kiosk hardware for zero-latency UI interactions, while the heavy AI processing is offloaded to a high-capacity remote GPU server on Vast.ai to ensure rapid image generation.

4.2 Modules design

- **Admin**
- **User**
- **AI Orchestrator**

4.3 Identification module description:

4.3.1 Admin

- ✓ **Garment Management**
Allows admins to add, edit, or delete items from the database.
- ✓ **Image Upload**
Stores actual garment variations to be fed to the AI engine.

4.3.2 Kiosk User

- ✓ **Camera Module**
Captures a high-quality image of the user.
- ✓ **Personalization Module**
Takes user inputs for Gender, Occasion, and Style to filter the catalog.
- ✓ **Catalog Browsing**
Displays recommended garments fetched from the database.
- ✓ **Result Viewer**
Displays the final generated image returned from the AI pipeline.

4.3.3 AI Orchestrator

✓ Workflow Patcher

Reads the user and garment images, then patches the internal ComfyUI workflow JSON dynamically to include the new images.

✓ Polling System

Constantly polls the cloud server to verify if processing is complete to immediately return the result to the UI.

4.4 Use case diagram:

The main aim of the system design is to explain the scenario using use case diagrams. Use case diagrams clarify the flow of the application by deriving the use cases for all the functionalities in form of diagrams for the students.

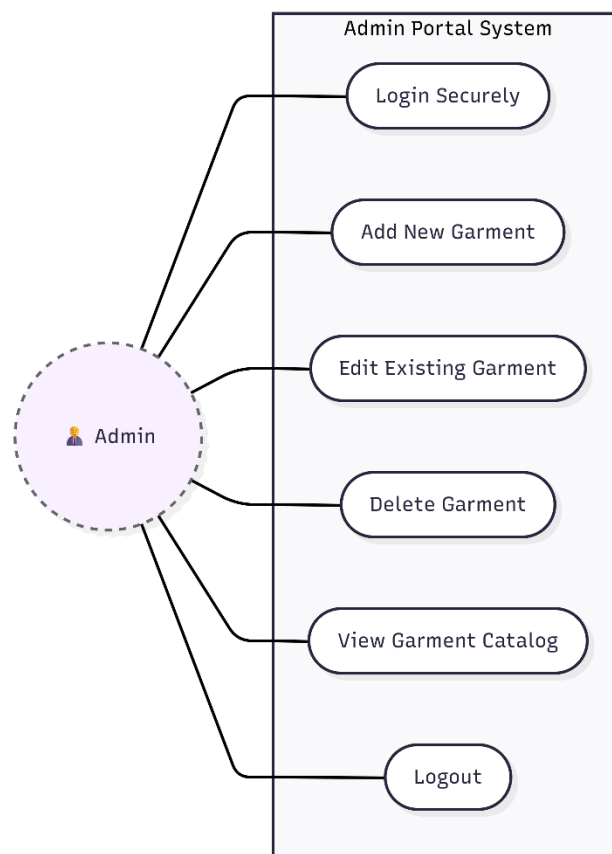


Figure 4.4.1 Admin

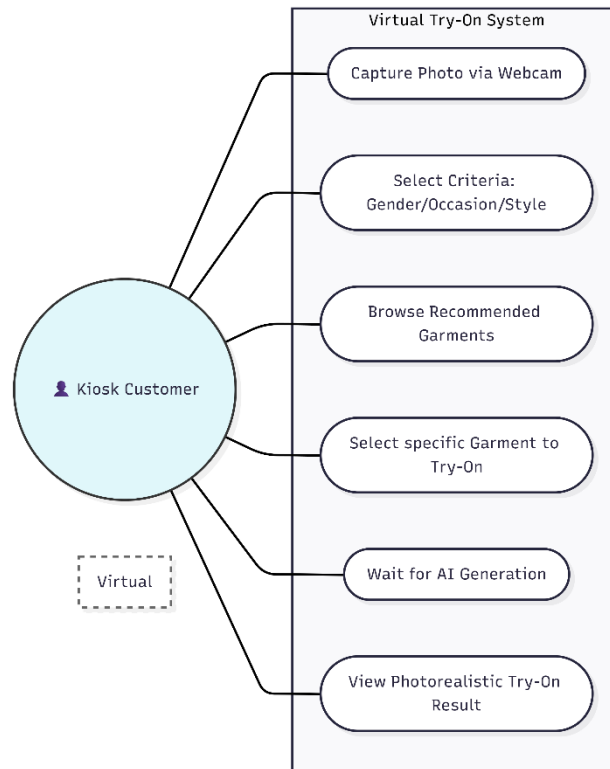


Figure 4.4.2 User

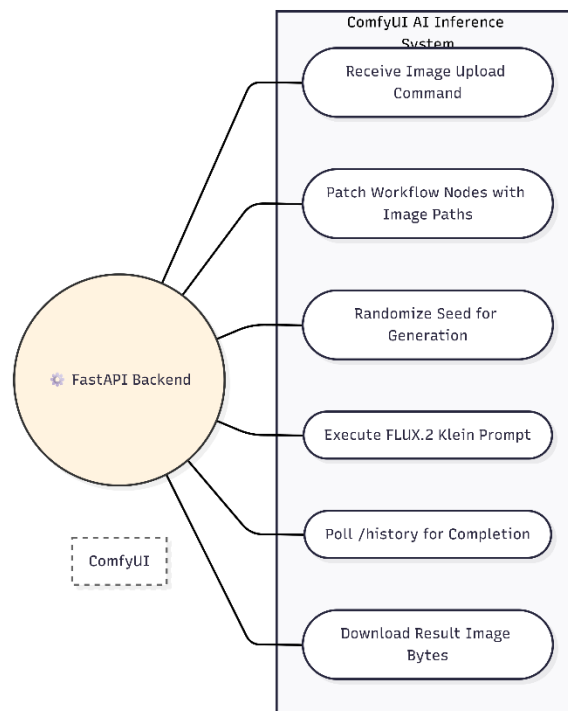


Figure 4.4.3 AI Orchestrator

4.5 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through a system. It is used to visualize how data moves from input to output, how it is processed, and where it is stored. DFDs help in understanding system functions and designing efficient workflows.

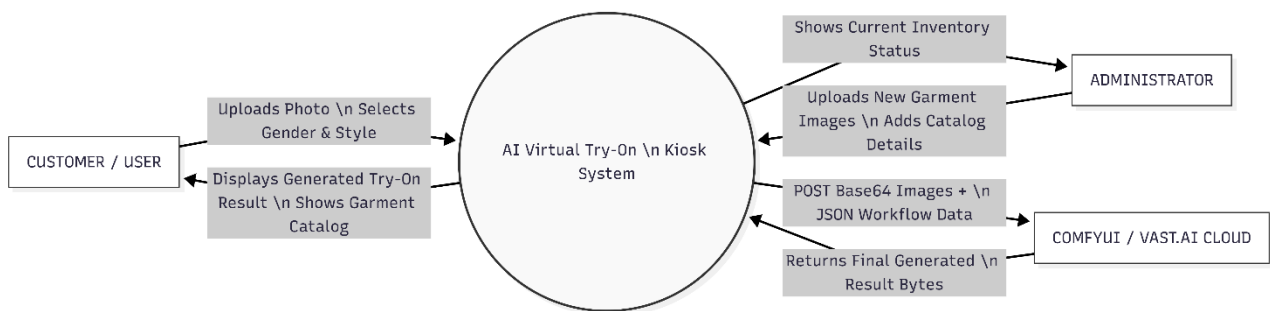


Figure 4.5.1

4.6 ENTITY RELATIONSHIP DIAGRAM

Entity Relationship Diagram (ERD) is a type of diagram used in database design that visually represents the structure of a database. It shows the entities (things you want to store information about), their attributes (details about those things), and the relationships between them.

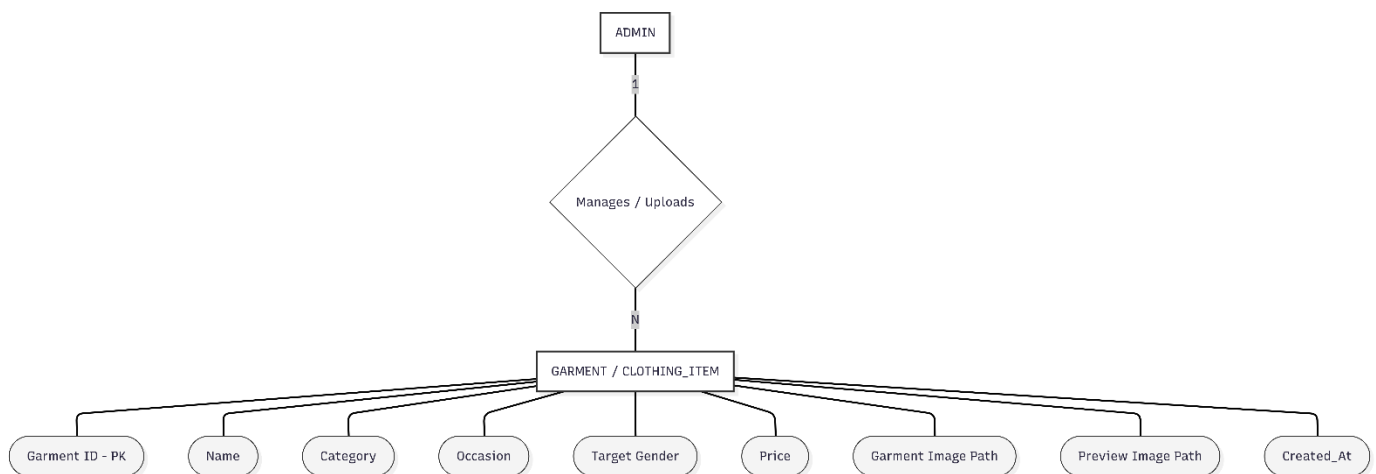


Figure 4.6.1

4.7 DATABASE DESIGN

4.7.1 Introduction

Database design is used to manage large bodies of information. The management of data involves both the definition of structure of storage and provision for the manipulation of information. In addition, the database system must provide the safety of the information solved despite system crashes or due to the attempts at unauthorized access. We have to fulfil certain conditions such as:

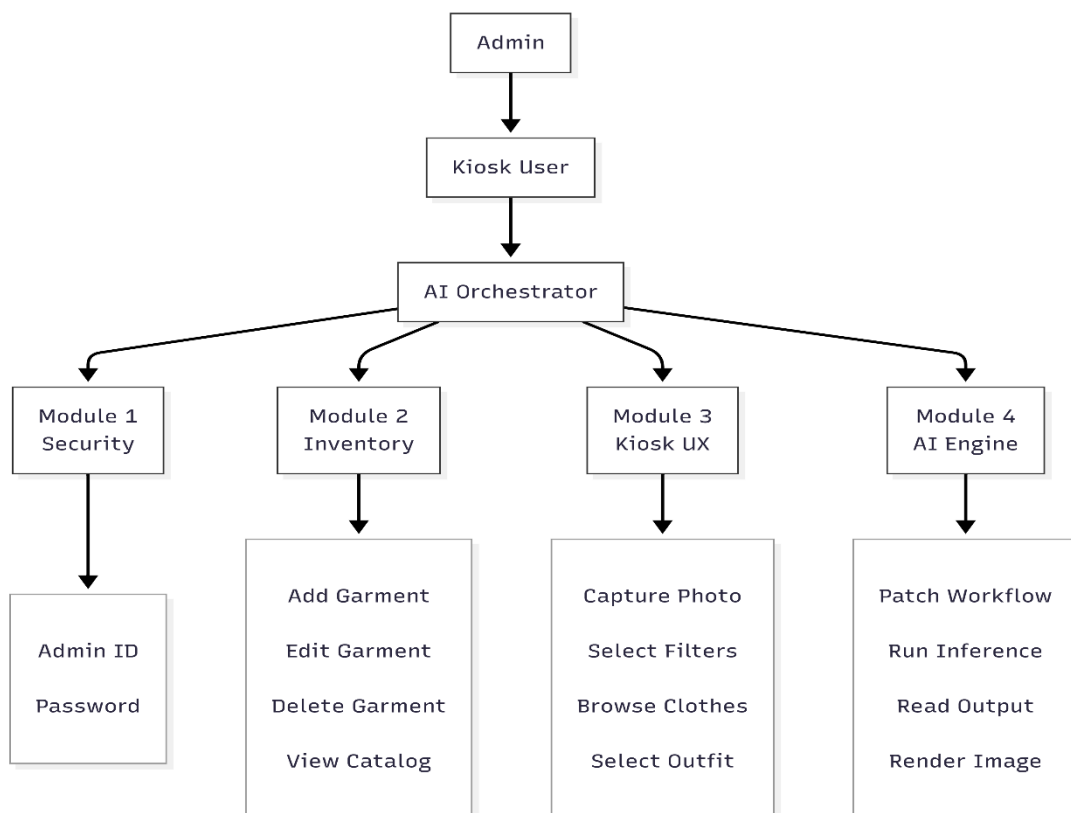
- Control Redundancy.
- Easy to use.
- Data Independence.
- Accuracy and Integrity.
- Recovery and Security.
- Performance

4.7.2 Database used in the module implementation

The system uses one database:

- SQLite

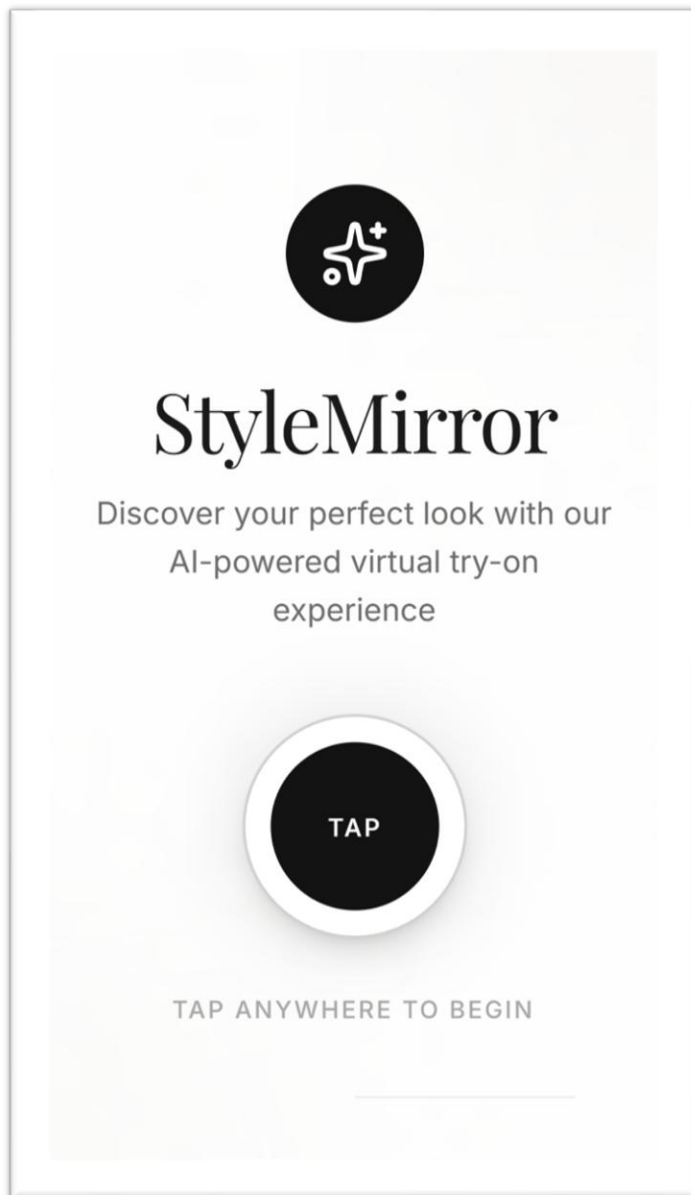
4.7.3 Pictorial representation of Database



CHAPTER - 5

SNAPSHOT

USER



STEP 2 OF 4

← Who are you shopping for?

Select an option so we can filter outfits best suited for you.



Male Clothes
Men's and Unisex



Female Clothes
Women's and Unisex



Continue →

STEP 2 OF 4

← Choose Your Occasion

What's the occasion? Let us curate the perfect look for you.



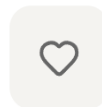
Business
Professional meetings & work



Casual
Everyday comfort & style



Party
Celebrations & nights out



Date Night
Romantic evenings



STEP 2 OF 4

←

Select Your Style

What aesthetic speaks to you?



Classic

Timeless elegance

☐



Modern

Contemporary edge

☒



Bohemian

Free-spirited flow

☐

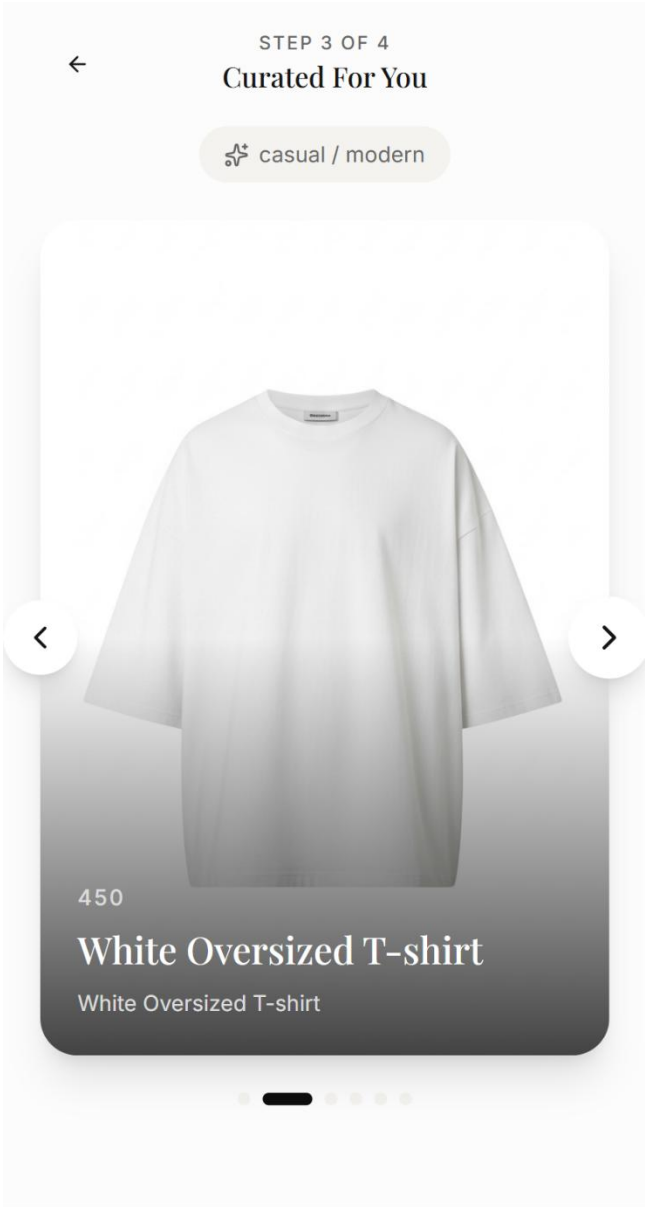


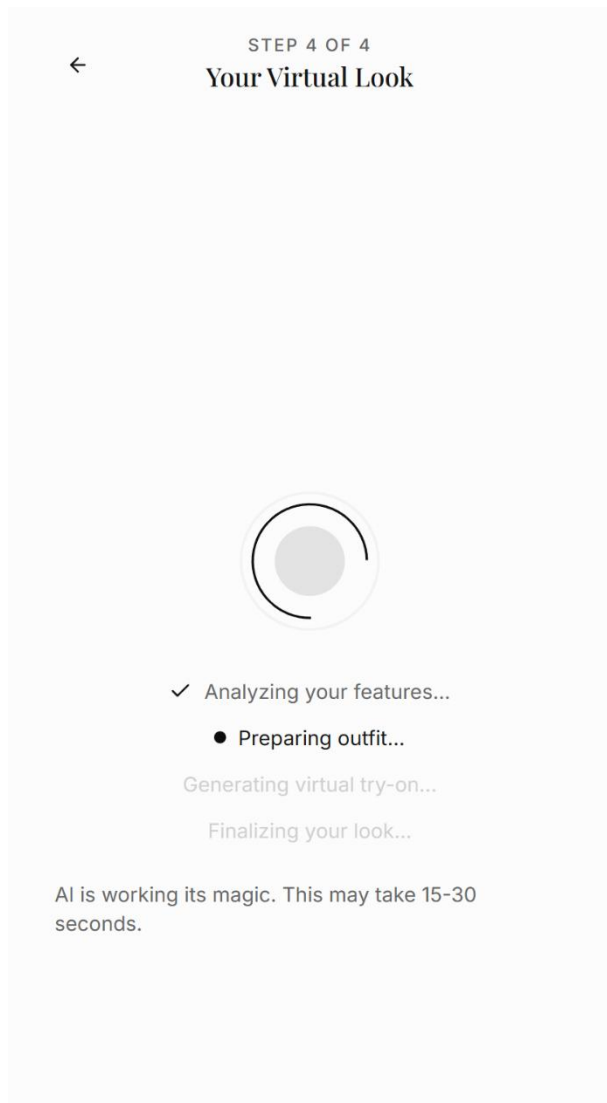
Minimalist

Clean & refined

☐

See Recommendations →







STEP 4 OF 4

Your Virtual Look

Split View

Overlay



White Oversized T-shirt

White Oversized T-shirt

450

Try Another



1 Issue ×

Complete Experience

ADMIN

Clothing Catalog

Manage clothing items for the virtual try-on kiosk

 Refresh

 Add Clothing



Hoodie

500

upper_full_sleeve • casual

modern

unisex

A Hoodie



White Oversized T-shirt

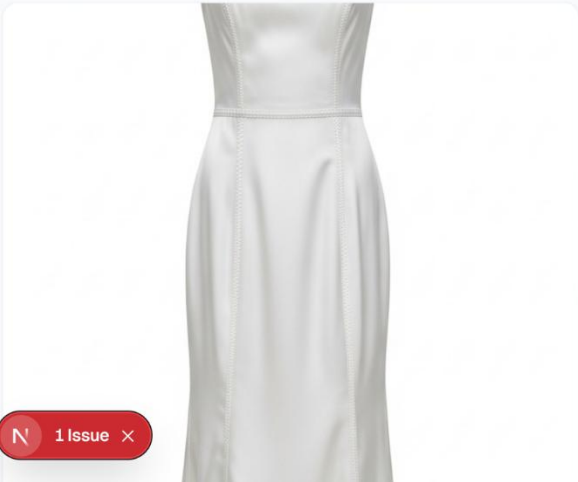
450

upper_half_sleeve • casual

modern

unisex

White Oversized T-shirt



Clothing Admin Portal

View Kiosk →

Clothing Catalog

Manage clothing items for the virtual try-on kiosk

Refresh

+ Add Clothing

Add New Clothing Item

Name *

e.g., Executive Blazer

Price

e.g., \$299

Category *

Upper Half Sleeve

Occasion *

Casual

Style *

Modern

Gender *

Unisex

Garment Description (IDM-VTON) *

e.g. A grey hooded sweatshirt with a front pocket

General Description

Describe the clothing item...

Garment Image * (for Try-On)

Upload a clean image of just the garment (flat lay or on mannequin)

Click to upload

Preview Image * (for UI display)

Upload a styled/lifestyle image for the catalog display

Click to upload

Cancel

Create

Hoodie

upper_full_sleeve

modern unisex

A Hoodie

450

N 1 Issue

24

CHAPTER - 6

SOURCE CODE

models.py

```
"""
SQLAlchemy ORM models for the database.
"""
import uuid
from datetime import datetime
from sqlalchemy import Column, String, DateTime, Text
from sqlalchemy.orm import declarative_base

Base = declarative_base()

def generate_uuid() -> str:
    """Generate a UUID string."""
    return str(uuid.uuid4())

class ClothingItem(Base):
    """Model for clothing items in the catalog."""

    __tablename__ = "clothing_items"

    id = Column(String(36), primary_key=True, default=generate_uuid)
    name = Column(String(255), nullable=False)
    category = Column(String(50), nullable=False) # shirt, pant, jacket, etc.
    occasion = Column(String(50), nullable=False) # business, casual, party, date
    style = Column(String(50), nullable=False) # classic, modern, minimalist
    gender = Column(String(20), nullable=False) # male, female, unisex
    garment_image = Column(String(500), nullable=False) # Path for ComfyUI try-on
    preview_image = Column(String(500), nullable=False) # Path for UI display grid
    description = Column(Text, nullable=True)
    price = Column(String(50), nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    def to_dict(self) -> dict:
        """Convert model instance to a plain Python dictionary."""
        return {
            "id": self.id,
            "name": self.name,
            "category": self.category,
            "occasion": self.occasion,
            "style": self.style,
```

```

    "gender": self.gender,
    "garment_image": self.garment_image,
    "preview_image": self.preview_image,
    "description": self.description,
    "price": self.price,
    "created_at": self.created_at.isoformat() if self.created_at else None,
    "updated_at": self.updated_at.isoformat() if self.updated_at else None,
}

```

tryon.py

```

"""
Try-On API routes — FLUX.2 Klein 9B cloth swap.
"""

from fastapi import APIRouter, HTTPException, Depends
from pydantic import BaseModel, Field
from sqlalchemy.orm import Session

from database import get_db, ClothingItem
from services.comfyui_client import comfyui_client

router = APIRouter(prefix="/try-on", tags=["Try-On"])

class TryOnRequest(BaseModel):
    person_image: str = Field(..., description="Base64 encoded person image from frontend")
    garment_id: str = Field(..., description="ID of the selected garment from the database")

class TryOnResponse(BaseModel):
    success: bool
    tryon_result_image: str | None = Field(None, description="Base64 encoded result image")
    error: str | None = None
    processing_time_ms: int | None = None

@router.post("", response_model=TryOnResponse)
async def generate_tryon(
    request: TryOnRequest,
    db: Session = Depends(get_db)
) -> TryOnResponse:
    """
    Generate a virtual try-on image using FLUX.2 Klein via ComfyUI.
    Only needs the person image (from webcam) and the garment ID (from DB).
    """
    # 1. Fetch garment image path from DB
    garment = db.query(ClothingItem).filter(ClothingItem.id == request.garment_id).first()
    if not garment:
        raise HTTPException(status_code=404, detail="Selected garment not found in database")

```

```

# 2. Send both images to the ComfyUI cloud pipeline
result = await comfyui_client.generate_tryon(
    person_image=request.person_image,
    garment_image=garment.garment_image,
)

if not result["success"]:
    raise HTTPException(
        status_code=500,
        detail=result.get("error", "ComfyUI generation failed"),
    )

return TryOnResponse(
    success=result["success"],
    tryon_result_image=result["tryon_result_image"],
    error=result.get("error"),
    processing_time_ms=result.get("processing_time_ms"),
)

```

comfyui_client.py

```

"""
ComfyUI client for FLUX.2 Klein 9B cloth swapping on Vast.ai GPU.
"""

import json, uuid, time, asyncio, base64
from pathlib import Path
from typing import Tuple, Dict, Any
import httpx
from config import settings

class ComfyUIClient:
    """Handles all communication with the remote ComfyUI GPU server."""

    def __init__(self):
        self.base_url = settings.COMFYUI_URL.rstrip("/")
        self.client_id = str(uuid.uuid4())
        self.workflow_path = settings.BASE_DIR.parent / "workflow_api.json"

    def _read_image(self, image_data: str) -> Tuple[bytes, str]:
        """Convert base64 data-URL or local file path into raw bytes."""
        if image_data.startswith("data:"):
            header, b64 = image_data.split(",", 1)
            mime = header.split(":")[1].split(";")[0]
            return base64.b64decode(b64), mime
        if image_data.startswith("/api/clothing/images/"):
            filepath = settings.GARMENTS_DIR / image_data.split("/")[-1]
        else:
            filepath = Path(image_data)

```

```

raw = filepath.read_bytes()
mime = "image/png" if str(filepath).endswith(".png") else "image/jpeg"
return raw, mime

async def _upload_image(self, image_data: str, prefix: str) -> str:
    """Upload an image to the remote ComfyUI /upload/image endpoint."""
    img_bytes, mime = self._read_image(image_data)
    ext = {"image/jpeg": ".jpg", "image/png": ".png"}.get(mime, ".jpg")
    filename = f"{prefix}_{uuid.uuid4().hex[:8]}{ext}"
    async with httpx.AsyncClient(timeout=60.0) as client:
        resp = await client.post(
            f"{self.base_url}/upload/image",
            data={"overwrite": "true"},
            files={"image": (filename, img_bytes, mime)},
        )
        resp.raise_for_status()
        return resp.json()["name"]

async def _download_image(self, filename: str, subfolder: str, img_type: str) -> str:
    """Download the output image from the GPU and return as base64 Data-URL."""
    async with httpx.AsyncClient(timeout=60.0) as client:
        resp = await client.get(
            f"{self.base_url}/view",
            params={"filename": filename, "type": img_type, "subfolder": subfolder},
        )
        resp.raise_for_status()
        mime = "image/jpeg" if resp.content[:2] == b"\xff\xd8" else "image/png"
        b64 = base64.b64encode(resp.content).decode("utf-8")
        return f"data:{mime};base64,{b64}"

async def generate_tryon(self, person_image: str, garment_image: str) -> dict:
    """
    Full pipeline:
    1. Upload person + garment images to cloud GPU.
    2. Patch workflow nodes 76 (person) and 81 (garment).
    3. Submit workflow to ComfyUI POST /prompt.
    4. Poll GET /history until image is ready.
    5. Download, encode as base64, return to FastAPI.
    """
    start = time.time()
    try:
        # Step 1: Upload images
        person_fn = await self._upload_image(person_image, "human")
        garment_fn = await self._upload_image(garment_image, "garment")

        # Step 2: Load and patch the workflow JSON
        with open(self.workflow_path, "r", encoding="utf-8") as f:
            workflow = json.load(f)
        nodes = {n["id"]: n for n in workflow.get("nodes", [])}
        if 76 in nodes: nodes[76]["widgets_values"][0] = person_fn
        if 81 in nodes: nodes[81]["widgets_values"][0] = garment_fn

```

```

if 145 in nodes: nodes[145]["widgets_values"][0] = int(time.time() * 1000) % (2**32)

# Step 3: Submit Job
api_prompt = self._build_api_prompt(workflow)
async with httpx.AsyncClient(timeout=30.0) as client:
    resp = await client.post(
        f'{self.base_url}/prompt',
        json={"prompt": api_prompt, "client_id": self.client_id},
    )
    resp.raise_for_status()
    prompt_id = resp.json()["prompt_id"]

# Step 4: Poll for result (max 3 minutes)
output_filename = output_subfolder = ""
output_type = "output"
async with httpx.AsyncClient(timeout=15.0) as client:
    for _ in range(90):
        await asyncio.sleep(2)
        hist = (await client.get(f'{self.base_url}/history/{prompt_id}')).json()
        if prompt_id in hist:
            for node_id in ["103", "117", "176"]:
                imgs = hist[prompt_id].get("outputs", {}).get(node_id, {}).get("images", [])
                if imgs:
                    output_filename = imgs[0]["filename"]
                    output_subfolder = imgs[0].get("subfolder", "")
                    output_type = imgs[0].get("type", "output")
                    break
            if output_filename:
                break

    if not output_filename:
        return {"success": False, "error": "Timed out (3 min). Check GPU instance.",
            "tryon_result_image": None}

# Step 5: Download and return
result_b64 = await self._download_image(output_filename, output_subfolder, output_type)
return {
    "success": True,
    "tryon_result_image": result_b64,
    "error": None,
    "processing_time_ms": int((time.time() - start) * 1000),
}

except Exception as exc:
    return {"success": False, "error": str(exc), "tryon_result_image": None,
        "processing_time_ms": int((time.time() - start) * 1000)}

comfyui_client = ComfyUIClient()

```

clothing.py

```
"""
Clothing management API routes.
"""

import base64, uuid
from typing import Optional
from pathlib import Path

from fastapi import APIRouter, Depends, HTTPException
from fastapi.responses import FileResponse
from sqlalchemy.orm import Session

from database import get_db, ClothingItem
from schemas import ClothingCreate, ClothingUpdate, ClothingResponse, ClothingListResponse
from config import settings

router = APIRouter(prefix="/clothing", tags=["Clothing"])

@router.get("/images/{filename}")
async def get_image(filename: str):
    """Serve a garment image file stored on the local server."""
    filepath = settings.GARMENTS_DIR / filename
    if not filepath.exists():
        raise HTTPException(status_code=404, detail="Image not found")
    return FileResponse(filepath)

@router.get("", response_model=ClothingListResponse)
async def list_clothing(
    occasion: Optional[str] = None,
    style: Optional[str] = None,
    gender: Optional[str] = None,
    category: Optional[str] = None,
    db: Session = Depends(get_db),
):
    """List all clothing items with optional filters."""
    query = db.query(ClothingItem)
    if occasion: query = query.filter(ClothingItem.occasion == occasion)
    if style: query = query.filter(ClothingItem.style == style)
    if gender: query = query.filter(ClothingItem.gender.in_([gender, "unisex"]))
    if category: query = query.filter(ClothingItem.category == category)
    items = query.all()
    return ClothingListResponse(
        items=[ClothingResponse.model_validate(i) for i in items],
        total=len(items),
    )

@router.get("/recommendations")
```

```

async def get_recommendations(
    occasion: Optional[str] = None,
    style: Optional[str] = None,
    gender: Optional[str] = None,
    limit: int = 6,
    db: Session = Depends(get_db),
):
    """Get curated garment recommendations for the kiosk."""
    query = db.query(ClothingItem)
    if occasion: query = query.filter(ClothingItem.occasion == occasion)
    if style: query = query.filter(ClothingItem.style == style)
    if gender: query = query.filter(ClothingItem.gender.in_([gender.lower(), "unisex"]))
    items = query.limit(limit).all()
    return ClothingListResponse(
        items=[ClothingResponse.model_validate(i) for i in items],
        total=len(items),
    )

```

```

@router.post("", response_model=ClothingResponse)
async def create_clothing(clothing: ClothingCreate, db: Session = Depends(get_db)):
    """Create a new clothing item (Admin panel)."""
    item = ClothingItem(**clothing.model_dump())
    db.add(item)
    db.commit()
    db.refresh(item)
    return ClothingResponse.model_validate(item)

```

```

@router.delete("/{item_id}")
async def delete_clothing(item_id: str, db: Session = Depends(get_db)):
    """Delete a clothing item and its image files (Admin panel)."""
    item = db.query(ClothingItem).filter(ClothingItem.id == item_id).first()
    if not item:
        raise HTTPException(status_code=404, detail="Clothing item not found")
    db.delete(item)
    db.commit()
    return {"success": True, "message": "Clothing item deleted"}

```

api.ts

```

/**
 * API client for communicating with the FastAPI backend.
 */

export const API_BASE_URL = process.env.NEXT_PUBLIC_API_URL ||
"http://localhost:8000/api";

/** Outfit maps directly to the backend ClothingItem model */

```

```

export interface Outfit {
  id:      string;
  name:    string;
  category: string;
  occasion: string;
  style:   string;
  gender:  string;
  garment_image: string; // Fed to ComfyUI for AI try-on processing
  preview_image: string; // Displayed in the kiosk catalog grid
  description: string | null;
  price:      string | null;
}

export interface TryOnRequest { person_image: string; garment_id: string; }
export interface TryOnResponse {
  success:      boolean;
  tryon_result_image: string | null;
  error:        string | null;
  processing_time_ms: number | null;
}

/** Fetch garment recommendations filtered by gender, occasion, and style */
export async function getRecommendations(
  occasion: string | null,
  style:   string | null,
  gender?: string | null
): Promise<Outfit[]> {
  const params = new URLSearchParams();
  if (occasion) params.append("occasion", occasion);
  if (style)   params.append("style", style);
  if (gender)  params.append("gender", gender);
  params.append("limit", "6");
  const res = await fetch(`${API_BASE_URL}/clothing/recommendations?${params}`);
  const data = await res.json();
  return data.items;
}

/** Send person image + garment ID to backend and get AI try-on result */
export async function generateTryOn(
  personImage: string,
  garmentId: string
): Promise<TryOnResponse> {
  const res = await fetch(`${API_BASE_URL}/try-on`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body:   JSON.stringify({ person_image: personImage, garment_id: garmentId }),
  });
  return await res.json() as TryOnResponse;
}

```


Admin

page.tsx

"use client"

```
import { useState, useEffect, useCallback } from "react"
import { Plus, Pencil, Trash2, Loader2, RefreshCw } from "lucide-react"
import { Button } from "@components/ui/button"
import { getAllClothing, createClothing, updateClothing, deleteClothing, type Outfit } from
"@/lib/api"
import { ClothingForm, type ClothingFormData } from "@components/admin/clothing-form"
import { ClothingCard } from "@components/admin/clothing-card"
```

```
export default function AdminPage() {
  const [clothing, setClothing] = useState<Outfit[]>([])
  const [isLoading, setIsLoading] = useState(true)
  const [error, setError] = useState<string | null>(null)
  const [showForm, setShowForm] = useState(false)
  const [editingItem, setEditingItem] = useState<Outfit | null>(null)
  const [isSubmitting, setIsSubmitting] = useState(false)
```

// Fetch all clothing items from the backend API

```
const fetchClothing = useCallback(async () => {
  setIsLoading(true)
  setError(null)
  try {
    const items = await getAllClothing()
    setClothing(items)
  } catch (err) {
    setError("Failed to load clothing items")
  } finally {
    setIsLoading(false)
  }
}, [])
```

```
useEffect(() => { fetchClothing() }, [fetchClothing])
```

// Handle creating a new garment

```
const handleCreate = async (data: ClothingFormData) => {
  setIsSubmitting(true)
  try {
    const result = await createClothing({
      name: data.name,
      category: data.category,
      occasion: data.occasion,
      style: data.style,
      gender: data.gender,
      garment_image: data.garmentImage,
      preview_image: data.previewImage,
```

```

        description: data.description,
        price: data.price,
    })
    if (result) { setShowForm(false); await fetchClothing() }
    else setError("Failed to create clothing item")
  } finally {
    setIsSubmitting(false)
  }
}

// Handle updating an existing garment
const handleUpdate = async (data: ClothingFormData) => {
  if (!editingItem) return
  setIsSubmitting(true)
  try {
    const updates: any = {
      name: data.name, category: data.category,
      occasion: data.occasion, style: data.style,
      gender: data.gender, description: data.description, price: data.price,
    }
    // Only send image fields if they have been changed
    if (data.garmentImage !== editingItem.garment_image)
      updates.garment_image = data.garmentImage
    if (data.previewImage !== editingItem.preview_image)
      updates.preview_image = data.previewImage

    const result = await updateClothing(editingItem.id, updates)
    if (result) { setEditingItem(null); setShowForm(false); await fetchClothing() }
    else setError("Failed to update clothing item")
  } finally {
    setIsSubmitting(false)
  }
}

// Handle deleting a garment
const handleDelete = async (id: string) => {
  if (!confirm("Are you sure you want to delete this item?")) return
  const success = await deleteClothing(id)
  if (success) await fetchClothing()
  else setError("Failed to delete clothing item")
}

return (
  <div className="space-y-6">
    { /* Header */ }
    <div className="flex items-center justify-between">
      <div>
        <h2 className="text-2xl font-bold text-slate-900">Clothing Catalog</h2>
        <p className="text-sm text-slate-600">
          Manage clothing items for the virtual try-on kiosk
        </p>

```

```

</div>
<div className="flex gap-2">
  <Button variant="outline" onClick={fetchClothing} disabled={isLoading}>
    <RefreshCw className={`h-4 w-4 mr-2 ${isLoading ? 'animate-spin' : ''}`} />
    Refresh
  </Button>
  <Button onClick={() => setShowForm(true)}>
    <Plus className="h-4 w-4 mr-2" /> Add Clothing
  </Button>
</div>
</div>

{/* Error Message */}
{error && (
  <div className="rounded-lg bg-red-50 border border-red-200 p-4 text-red-700">
    {error}
    <button onClick={() => setError(null)} className="ml-4 text-sm underline">
      Dismiss
    </button>
  </div>
)}

{/* Add / Edit Form Modal */}
{showForm && (
  <div className="fixed inset-0 z-50 flex items-center justify-center bg-black/50">
    <div className="max-h-[90vh] w-full max-w-2xl overflow-auto rounded-lg bg-white
p-6 shadow-xl">
      <h3 className="mb-4 text-lg font-semibold">
        {editingItem ? "Edit Clothing Item" : "Add New Clothing Item"}
      </h3>
      <ClothingForm
        initialData={editingItem ? {
          name: editingItem.name,
          category: editingItem.category,
          occasion: editingItem.occasion,
          style: editingItem.style,
          gender: editingItem.gender,
          garmentImage: editingItem.garment_image,
          previewImage: editingItem.preview_image,
          description: editingItem.description || "",
          price: editingItem.price || "",
        } : undefined}
        onSubmit={editingItem ? handleUpdate : handleCreate}
        onCancel={() => { setShowForm(false); setEditingItem(null) }}
        isSubmitting={isSubmitting}
      />
    </div>
  </div>
)}

{/* Loading Spinner */}

```

```

    {isLoading && (
      <div className="flex items-center justify-center py-12">
        <Loader2 className="h-8 w-8 animate-spin text-slate-400" />
      </div>
    )}

    {/* Clothing Grid */}
    {!isLoading && clothing.length > 0 && (
      <div className="grid gap-6 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4">
        {clothing.map((item) => (
          <ClothingCard
            key={item.id}
            item={item}
            onEdit={() => { setEditingItem(item); setShowForm(true) }}
            onDelete={() => handleDelete(item.id)}
          />
        ))}
      </div>
    )}
  </div>
)
}

```

main.py

```

"""
FastAPI Backend for Virtual Try-On Kiosk
Main application entry point.
"""

from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from config import settings
from database import init_db
from routes import clothing_router, tryon_router

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Runs on startup and shutdown of the server."""
    # --- Startup ---
    print("🚀 Starting Virtual Try-On Backend...")
    init_db() # Create DB tables if they don't exist
    print("✅ Database initialized")
    print(f"📁 Storage directory: {settings.STORAGE_DIR}")

    yield # App runs here

```

```

# --- Shutdown ---
print("👋 Shutting down...")

# Create the main FastAPI app instance
app = FastAPI(
    title="Virtual Try-On Kiosk API",
    description="Backend API for the virtual try-on kiosk application",
    version="1.0.0",
    lifespan=lifespan,
)

# Allow cross-origin requests from the Next.js frontend
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.CORS_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Register API route modules
app.include_router(clothing_router, prefix=settings.API_PREFIX)
app.include_router(tryon_router, prefix=settings.API_PREFIX)

@app.get("/")
async def root():
    """Health check — confirms the server is running."""
    return {
        "status": "healthy",
        "service": "Virtual Try-On Kiosk API",
        "version": "1.0.0",
    }

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)

```

config.py

```

"""
Configuration settings for the backend application.
"""

from pathlib import Path
from pydantic_settings import BaseSettings

```

```

class Settings(BaseSettings):
    """All application settings — can be overridden via .env file."""

    # Base directory paths
    BASE_DIR: Path = Path(__file__).parent
    STORAGE_DIR: Path = BASE_DIR / "storage"
    GARMENTS_DIR: Path = STORAGE_DIR / "garments"

    # SQLite Database
    DATABASE_URL: str = "sqlite:///./database/kiosk.db"

    # ComfyUI Cloud GPU Instance (Vast.ai)
    # Change this URL every time you rent a new GPU instance
    COMFYUI_URL: str = "http://127.0.0.1:8188"

    # API Settings
    API_PREFIX: str = "/api"
    CORS_ORIGINS: list[str] = [
        "http://localhost:3000", # Next.js dev server
        "http://127.0.0.1:3000",
    ]

    # File upload size limit
    MAX_FILE_SIZE: int = 10 * 1024 * 1024 # 10 MB

    class Config:
        env_file = ".env" # Automatically loads COMFYUI_URL from .env
        extra = "ignore"

```

```

settings = Settings()

```

```

# Auto-create storage directories on first run
settings.STORAGE_DIR.mkdir(parents=True, exist_ok=True)
settings.GARMENTS_DIR.mkdir(parents=True, exist_ok=True)

```

session-context.tsx

```

"use client"

```

```

import { createContext, useContext, useState, useCallback, useEffect, type ReactNode } from
"react"

```

```

// All possible screens in the kiosk flow

```

```

export type KioskScreen =

```

```

| "welcome"
| "camera"
| "preferences"
| "recommendations"
| "try-on"
| "exit"

```

```

// Data stored across the entire customer session
export interface SessionData {
  capturedImage: string | null // Webcam photo of the customer
  selectedOccasion: string | null // e.g. casual, business, party
  selectedStyle: string | null // e.g. classic, modern, minimalist
  selectedOutfit: Outfit | null // The garment the customer chose
  tryOnResult: string | null // Base64 AI-generated result image
  detectedGender: string | null // Gender selected by the customer
}

interface SessionContextType {
  currentScreen: KioskScreen
  sessionData: SessionData
  sessionId: string | null
  isSessionActive: boolean
  startSession: () => void
  endSession: () => void
  navigateTo: (screen: KioskScreen) => void
  updateSessionData: (data: Partial<SessionData>) => void
  resetSession: () => void
}

const SESSION_TIMEOUT = 5 * 60 * 1000 // 5 minutes — full session limit
const INACTIVITY_TIMEOUT = 2 * 60 * 1000 // 2 minutes — auto-reset if customer walks away

const SessionContext = createContext<SessionContextType | undefined>(undefined)

export function SessionProvider({ children }: { children: ReactNode }) {
  const [currentScreen, setCurrentScreen] = useState<KioskScreen>("welcome")
  const [sessionData, setSessionData] = useState<SessionData>({
    capturedImage: null, selectedOccasion: null,
    selectedStyle: null, selectedOutfit: null,
    tryOnResult: null, detectedGender: null,
  })
  const [sessionId, setSessionId] = useState<string | null>(null)
  const [isSessionActive, setIsSessionActive] = useState(false)
  const [lastActivity, setLastActivity] = useState<number>(Date.now())

  const resetSession = useCallback(() => {
    setCurrentScreen("welcome")
    setSessionData({ capturedImage: null, selectedOccasion: null,
      selectedStyle: null, selectedOutfit: null,
      tryOnResult: null, detectedGender: null })
    setSessionId(null)
    setIsSessionActive(false)
  }, [])

  const startSession = useCallback(() => {
    setSessionId(`session_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`)
    setIsSessionActive(true)
  }, [])

```

```

    setLastActivity(Date.now())
    setCurrentScreen("camera")
  }, [])

const endSession = useCallback(() => {
  setCurrentScreen("exit")
  setTimeout(() => resetSession(), 5000) // Show exit screen for 5s, then reset
}, [resetSession])

const navigateTo = useCallback((screen: KioskScreen) => {
  setCurrentScreen(screen); setLastActivity(Date.now())
}, [])

const updateSessionData = useCallback((data: Partial<SessionData>) => {
  setSessionData((prev) => ({ ...prev, ...data })); setLastActivity(Date.now())
}, [])

// Auto-end session after 5 minutes
useEffect(() => {
  if (!isSessionActive) return
  const t = setTimeout(() => endSession(), SESSION_TIMEOUT)
  return () => clearTimeout(t)
}, [isSessionActive, endSession])

// Auto-reset if customer is inactive for 2 minutes
useEffect(() => {
  if (!isSessionActive) return
  const t = setInterval(() => {
    if (Date.now() - lastActivity > INACTIVITY_TIMEOUT) endSession()
  }, 10000)
  return () => clearInterval(t)
}, [isSessionActive, lastActivity, endSession])

// Track any click or touch as activity
useEffect(() => {
  if (!isSessionActive) return
  const handler = () => setLastActivity(Date.now())
  window.addEventListener("touchstart", handler)
  window.addEventListener("click", handler)
  return () => {
    window.removeEventListener("touchstart", handler)
    window.removeEventListener("click", handler)
  }
}, [isSessionActive])

return (
  <SessionContext.Provider value={{
    currentScreen, sessionData, sessionId, isSessionActive,
    startSession, endSession, navigateTo, updateSessionData, resetSession,
  }}>
    {children}
  </SessionContext.Provider>
)

```



```
    </SessionContext.Provider>
  )
}

export function useSession() {
  const context = useContext(SessionContext)
  if (!context) throw new Error("useSession must be used within a SessionProvider")
  return context
}
```

CHAPTER 7

CONCLUSION

The AI-powered Virtual Try-On Kiosk is a fully developed end-to-end solution that transforms the retail fashion experience by replacing traditional trial rooms with an intelligent digital system. Built using Next.js, FastAPI, SQLite, and the FLUX.2 Klein 9B generative AI model running on a cloud GPU via ComfyUI, the system enables customers to capture their image, select preferences, browse garments, and instantly visualize photorealistic outfit changes in real time. It also includes an admin panel for efficient inventory management. Overall, the project demonstrates the effective integration of generative AI, cloud computing, and full-stack development to solve real-world retail challenges while enhancing customer convenience and engagement.

7.1 Future Enhancement

- Size recommendation based on body measurements
- Multi-angle try-on — front, side, and back view
- WhatsApp / Email sharing of the try-on result
- Customer session history with loyalty card integration
- Support for accessories — watches, glasses, and handbags
- Live video overlay try-on (Augmented Reality mode)
- On-device AI inference for offline and privacy-safe operation
- POS (Point of Sale) billing system integration
- Multi-language support — Hindi, Kannada, Tamil, and more
- Analytics dashboard — popular styles, peak hours, conversion rates
- WebSocket-based real-time generation progress bar
- Image super-resolution post-processing for sharper results
- Dedicated GPU server to eliminate cloud cold-start delays

CHAPTER - 8

BIBLIOGRAPHY

1. FLUX.2 Klein 9B Model — Hugging Face Model Hub <https://huggingface.co/black-forest-labs/FLUX.1-dev>
2. ComfyUI — Open Source AI Generation Engine <https://github.com/comfyanonymous/ComfyUI>
3. FastAPI — Modern Python Web Framework Documentation <https://fastapi.tiangolo.com/>
4. Next.js 14 — React Framework Official Documentation <https://nextjs.org/docs>
5. SQLite — Lightweight Relational Database Documentation <https://www.sqlite.org/docs.html>
6. SQLAlchemy — Python ORM Documentation <https://docs.sqlalchemy.org/>
7. Pydantic — Data Validation Library for Python <https://docs.pydantic.dev/>
8. Vast.ai — GPU Cloud Compute Platform <https://vast.ai/>
9. Tailwind CSS — Utility-First CSS Framework <https://tailwindcss.com/docs>
10. React.js — Frontend JavaScript Library <https://react.dev/>
11. Framer Motion — Animation Library for React <https://www.framer.com/motion/>
12. HTTPX — Async HTTP Client for Python <https://www.python-httpx.org/>
13. Low-Rank Adaptation (LoRA) — Research Paper Hu et al., 2021 — <https://arxiv.org/abs/2106.09685>
14. Virtual Try-On Research — VITON-HD <https://arxiv.org/abs/2103.16874>